

NomElemento **In** *NomConjunto*

donde *NomElemento* es el nombre del elemento sobre el que se inquiera y *NomConjunto* el conjunto. El resultado de esta operación es un dato **Boolean**, y se evalúa a **True** sólo si el elemento pertenece al conjunto.

En el programa **Tres** también se ilustra la asignación de valores a los conjuntos, si bien falta añadir que también se puede asignar el conjunto vacío (`[]`).

Los operadores aritméticos y lógicos cuando actúan entre conjuntos tienen un significado nuevo. Si *A*, *B* y *C* son conjuntos de elementos del mismo tipo, las operaciones permitidas son :

<code>C := A + B</code>	{ <i>C</i> : conjunto union de <i>A</i> y <i>B</i> }
<code>C := A * B</code>	{ <i>C</i> : conjunto interseccion de <i>A</i> y <i>B</i> }
<code>C := A - B</code>	{ <i>C</i> : conjunto diferencia de <i>A</i> y <i>B</i> }

También se pueden comparar los conjuntos con los operadores lógicos dando lugar a un dato **Boolean**. Si *A* y *B* son dos conjuntos de elementos del mismo tipo y *verdad* un dato **Boolean**, las comparaciones siguientes son posibles:

<code>verdad := A = B</code>	{ <i>verdad</i> es True si <i>A</i> y <i>B</i> son iguales}
<code>verdad := A <> B</code>	{ <i>verdad</i> es True si <i>A</i> y <i>B</i> son distintos}
<code>verdad := A <= B</code>	{ <i>verdad</i> es True si <i>A</i> es subconjunto de <i>B</i> }
<code>verdad := A => B</code>	{ <i>verdad</i> es True si <i>B</i> es subconjunto de <i>A</i> }

teniendo en cuenta que el conjunto vacío es subconjunto de todo conjunto. También hay que tener en cuenta que el número de elementos que pueden formar un conjunto en *PASCAL* está limitado a un máximo de 256.

9.4 Arrays

Cuando se tiene que seguir la pista a un grupo o de datos es muy útil referirse a todos ellos con un mismo nombre y distinguir entre los elementos mediante el lugar que ocupan en el grupo. En realidad se trata de ampliar la conveniencia de los conjuntos estudiados anteriormente a grupos de datos de cualquier tipo. En el **Array** los elementos del grupo se ordenan asignándose a cada elemento un número de orden o dirección. El tipo de dato **Array** (tabla) es uno de los datos con estructura más importantes de un lenguaje. Esto es así, porque corresponde a una ordenación de datos similar a la que se

realiza en la memoria central del ordenador. Se puede pensar que un **Array** es un conjunto de celdillas contiguas, en cada una de ellas se puede almacenar un dato simple, y cada dato simple está identificado por la posición que ocupa en esa tabla. La ventaja de esta estructura es que se tarda el mismo tiempo en acceder a cualquier elemento, ya que se accede a él exclusivamente por su dirección. La desventaja, es que se trata de una estructura estática en el sentido de que la longitud máxima de la tabla se ha de especificar a priori. Para definir el tipo de dato **Array** se utiliza la siguiente sintaxis:

Array [*rango*] **Of** *tipo*

donde *tipo* es el tipo de datos que se almacena en cada una de las posiciones de la tabla y *rango* la especificación del rango de variación del índice (entero) que identifica las posiciones de los elementos de la tabla. Este rango se concreta especificando el primer y último índice separados por dos puntos consecutivos. Sólo se permite un rango consecutivo.

La especificación de **Array** puede aparecer tanto en una definición de tipo de dato, por ejemplo,

Type

```
vector = Array [1..30] Of Real;
```

o directamente en la declaración de una variable, por ejemplo,

Var

```
texto : Array [0..3000] Of Char;
```

El índice que sirve para especificar la posición de los elementos no tiene porque empezar en 0 o 1. Si para el programa tiene sentido utilizar un rango como [127..345], es perfectamente válido. Para acceder a un elemento de un **Array** se utiliza el identificador seguido de su índice entre paréntesis cuadrados.

En el programa **Primos** tenemos un ejemplo de utilización de esta estructura de datos. Se trata de la determinación de números primos utilizando el famoso algoritmo de la criba de Eratóstenes.

```
Program Primos;
```

```
Const
```

```
MAX = 1000;
```

```
{ Calculo de los numeros primos entre 1 y MAX
```

```
  utilizando el algoritmo de la Criba de Eratostenes }
```

```

Var
  esPrimo : Array [1..MAX] of Boolean;
  i,j : Integer;
Begin { primos }
  {Se inicializa la tabla}
  esPrimo[1] := False;
  For i := 2 to MAX Do esPrimo[i] := True;
  {En la criba de Eratostenes se parte de todos los numeros
   y se van eliminando todos los multiplos de los primos
   elegidos. Seran primos elegidos aquellos que , en orden
   ascendente, no hayan sido marcados como multiplos }

  For i := 2 To MAX Div 2 Do
    For j := 2 To MAX Div i Do
      esPrimo[i*j] := False;
  {Se cuenta el numero de primos encontrados }
  j:= 0; For i := 1 To MAX Do If esPrimo[i] Then j := j + 1;

  Writeln('Entre 1 y ',MAX:5,' se han encontrado ', j:3,' primos');

  For i := 1 To MAX Do If esPrimo[i] Then Write(i:4);
End. { primos }

```

Se define el **Array** `esPrimo` para almacenar datos **Boolean** que permiten seguir la pista de los números que son múltiplos de otros. En este programa se hace un uso apropiado de la ventaja ofrecida por la estructura de dato **Array**. Se utiliza un tiempo constante para acceder a cualquier elemento de la tabla, independientemente de que se trate del primero o el último. Además se puede incluir en el programa una relación ventajosa entre la posición de un elemento en la tabla y su significado.

La estructura **Array** se usa muy a menudo en el cálculo científico, pues se trata de la realización de un vector, si cada elemento de la tabla se interpreta como un componente del vector. Igualmente, se pueden definir tablas de tablas (**Array** bidimensional) para manipular matrices, y **Array** multidimensionales para los tensores.

La especificación del rango, cuando se declaran, en los **Array** multidimensionales se puede realizar separando con comas los rangos de cada una de las dimensiones. Igualmente, el acceso al contenido de una posición de

estas tablas múltiples se puede realizar separando con comas los índices de cada una de las dimensiones.

El siguiente procedimiento **MultMatriz** se puede utilizar para multiplicar matrices.

```

Program Cuatro;
  Const
    MAXDIM = 20;
  Type
    Numeros = Real;
    Matriz = Array[1..MAXDIM,1..MAXDIM] Of Numeros;
  Var
    n : Integer;
    A,B,C : Matriz;
  Procedure Leematriz ( Var X : Matriz); FORWARD;
  Procedure Escribematriz ( X : Matriz); FORWARD;
  Procedure MultMatriz ( dim : Integer;
                        Var
                          A1 , A2 , M : Matriz );
  Var
    i,j,k : Integer;
    x : Numeros;
  Begin { MultMatriz }
    For i := 1 To dim Do
      For j := 1 To dim Do
        Begin
          x:= 0.0;
          For k := 1 To dim Do
            x := x + A1 [i,k] * A2 [k,j];
          M[i,j] := x;
        End; {End del doble For}
      End; { MultMatriz }
    { Aqui se encontrarian las definiciones de los }
    { dos procedimientos que se han omitido }
  Begin
    Writeln('Dimension de las matrices: ');
    Readln(n);
    Leematriz(A); Leematriz(B);
  
```

```

MultMatriz(n,A,B,C);
Writeln('La Matriz producto es:');
Escribematriz(C);
End.

```

Pero la definición de los **Array** multidimensionales también se puede hacer como una tabla de tablas. Por ejemplo, el tipo de dato **Matriz** se podría haber definido del siguiente modo:

```
Matriz = Array[1..MAXDIM] Of Array [1..MAXDIM] Of Numeros;
```

e igualmente los componentes de este tipo de datos se pueden referir como:

```
x := x + A1 [i][k] * A2 [k][j];
```

Ambas definiciones y utilizaciones son equivalentes. Las limitaciones de las estructuras **Array** definidas de este modo están en la capacidad de almacenamiento. Una estructura **Array** no puede ocupar toda la memoria disponible en el ordenador por muchos motivos que se estudiarán en el tema de gestión de memoria. Basta con uno de ellos: en *TURBO PASCAL* no se puede crear un Array que ocupe más de un segmento de memoria ($2^{16} = 65,536$ bytes).

La estructura de datos **String** es simplemente un caso especial de tabla de datos tipo **Char** en el que se reserva la posición de índice cero para almacenar el carácter correspondiente al número que indica los elementos almacenados. Ya vimos en su día que el tipo de datos **String** tiene su capacidad limitada a 255 caracteres. El programador puede muy fácilmente definir un tipo de dato similar al **String** con capacidad superior. En este caso, tendrá que definir sus propias funciones para manipularlos. La siguiente función **Concatena** podría ser la función que uniera dos de estas cadenas de caracteres.

```

Program Cinco;
Const
  MAXDIM = 300;
  { Los caracteres se guardan en enteros como su ordinal }
Type
  Cadena = Array[0..MAXDIM] Of Integer;
Var
  A,B : Cadena;
Procedure LeeCadena (Var x : Cadena);

```

```

Var
  i : Integer;
  c : Char;
Begin { LeeCadena }
  i:= 1;
  Repeat
    Read(c);
    x[i] := Ord(c);
    i := i+1;
  Until ( EOLN );
  x[0] := i - 1;
  { Lee los caracteres ASCII 13 (CR) y 10 (LF)
    que delimitan el fin de linea }
  Read(c);Read(c);
End; { LeeCadena }

Procedure EscribeCadena ( x : Cadena);
Var
  i: Integer;
Begin { EscribeCadena }
  For i := 1 to x[0] Do Write(Chr( x[i] ));
End; { EscribeCadena }

{ Procedimiento para unir a la cadena A1 la cadena A2 }
Procedure Concatena ( Var
                      A1 , A2 : Cadena );

Var
  dim1 , i : Integer;
Begin
  dim1 := A1[0];
  For i := 1 To A2[0] Do
    A1[ dim1 + i ] := A2[i];
  A1[0] := dim1 + A2[0];
End;

Begin
  Writeln('');

```

```

LeeCadena(A);
LeeCadena(B);
Concatena(A,B);
Writeln('');
EscribeCadena(A);
End.

```

Se aprecia en la función **Concatena** que el hecho de poder acceder a la longitud de la cadena permite escribir algoritmos muy eficientes de cadenas de caracteres. Sin embargo, esta estructura definida para las cadenas de caracteres es ineficiente en cuanto almacenamiento. Se están almacenando los caracteres ASCII en 2 bytes mientras que sería suficiente uno sólo.

9.5 Registros

La situación encontrada en el problema planteado anteriormente de manipulación de cadenas es muy común. Lo más frecuente es encontrarse en una situación en la que se quiere representar en un tipo de dato una información que posee estructura interna y es heterogénea, es decir, los campos en los que se subdivide el dato no son todos del mismo tipo. Para ello el *PASCAL*, como otros lenguajes de programación actuales, suministra los datos tipo registro (en inglés *records*).

Un registro es un tipo de dato definido por el programador en el que puede especificar su estructura interna.

El programador da nombre al nuevo tipo de dato y a cada uno de los campos que lo componen, y especifica el tipo de dato que puede ocupar cada uno de los campos. La sintaxis para estas especificaciones es:

```

Type
    nombre =
        Record
            NombreCampo1 : TipoDato1 ;
            NombreCampo2 : TipoDato2 ;
            . . . . .
        End;

```

donde *nombre* es el identificador elegido para el registro, y *NombreCampo1*, *NombreCampo2*, ... son los identificadores elegidos para los distintos